

A Flexible MILP Formulation for Battery-Constrained Drone Filming with Time Windows

Jaime Díaz-Trechuelo Sánchez-Moliní Inmaculada Ventura Molina
José Miguel Díaz-Báñez

September 2026

Abstract

We study the offline planning of a single cinematographic drone that films spatially distributed events during prescribed time windows. The drone may perform several base-to-base trips, each subject to a battery-endurance limit, and the objective is to maximize total filming time. Existing graph-based methods discretize time and exploit a specialized dynamic program. They are attractive for large, stable instances, but the associated state graph and recurrence are closely tied to the original assumptions: new operational requirements may require a new state space, new dominance arguments, and a new implementation. We develop instead a mixed-integer linear programming (MILP) formulation intended for small and medium instances, where flexibility, exact certificates, and rapid model modification can matter more than worst-case scalability. We prove safe bounds on the number of useful trips, justify an event-splitting rule by a minimum-excursion argument, show that temporal precedence eliminates disconnected subtours, and establish the correspondence between feasible integer solutions and schedules in the split-node model. We also discuss graph pruning, tight big- M constants, coverage incentives, and the trade-off between a bespoke polynomial algorithm and a general-purpose exact model.

Keywords: aerial cinematography; drone routing; time windows; battery constraints; mixed-integer linear programming; node splitting.

1 Introduction

Outdoor events such as cycling stages, road races, and live performances contain moments that are known in advance to occur at particular places and during particular time intervals. A media director may therefore specify a collection of filming demands, each consisting of a location and a time window. A drone can record a demand only while it is at the corresponding location and the window is active. Travel consumes time and battery, and the drone must periodically return to a base station to replace or recharge its battery.

This setting resembles the traveling salesman problem with time windows (TSPTW) and the vehicle-routing problem with time windows (VRPTW), but it differs in several important ways. Service is optional; its duration is a decision rather than fixed data; the same event may be left and revisited; and the schedule comprises an unknown number of consecutive base-to-base trips. The natural objective is not the number of demands served or the distance traveled, but the total time filmed.

Caraballo et al. [1] formulate a broader cinematographic planning problem with moving tasks and multiple drones. Their method discretizes the task trajectories in time, constructs a directed acyclic graph, and uses dynamic programming to find an optimal solution of the

discretized one-drone problem. Multiple drones are handled by a sequential greedy strategy. Related work shows that the continuous problem is polynomially solvable for one drone under the geometric assumptions considered there, whereas the limited-battery problem becomes NP-complete for a team of drones [2, 3].

Those complexity results do not settle which computational approach is preferable in a particular application. A specialized polynomial algorithm may be the right choice for a large, repeatedly solved problem whose rules are stable. Its implementation, however, is not merely the statement of a recurrence. One must construct and maintain a time-expanded graph, encode battery states, prove that the retained transitions are sufficient, implement reconstruction and tie-breaking, and revise these components whenever a new requirement changes the state description. In contrast, a MILP model delegates much of the search, bounding, and certification machinery to a mature solver. Its worst-case running time is exponential, but for modest instances it can be easier to validate, extend, and use as an exact benchmark for heuristics.

The contribution of this paper is a rigorous MILP treatment of the static, one-drone case. In particular, we:

- (i) state the modeling assumptions and the graph-based dynamic-programming baseline;
- (ii) identify precisely why that algorithm is structurally rigid under new demands;
- (iii) prove a safe upper bound on the number of useful trips;
- (iv) prove a minimum-excursion result that justifies splitting long event windows;
- (v) give a complete arc-flow MILP with time, service, battery, and sequencing variables;
- (vi) prove the absence of positive-time subtours and the validity of the formulation; and
- (vii) show how common operational extensions can be expressed with a few linear constraints.

The emphasis is methodological. The split-node model is a controlled finite representation of a continuous-time problem; it should not be confused with an assertion that one fixed splitting rule is exact for every possible application. We state explicitly which conclusions are exact and which are modeling choices.

2 Problem statement

Let the planning horizon be $[0, T]$. There is one base station, indexed by 0, and a set $\mathcal{K} = \{1, \dots, K\}$ of stationary filming locations. Event $i \in \mathcal{K}$ is available during the closed interval

$$I_i = [t_i, t_i^*], \quad 0 \leq t_i < t_i^* \leq T.$$

Let $D_{ij} \geq 0$ be the distance from location i to location j , where $i, j \in \{0\} \cup \mathcal{K}$. We assume that D is a metric: $D_{ii} = 0$, $D_{ij} = D_{ji}$, and

$$D_{ik} \leq D_{ij} + D_{jk} \quad \text{for all } i, j, k \in \{0\} \cup \mathcal{K}.$$

The drone travels at constant speed, normalized to one, so distance and travel time have the same numerical value. Equivalently, D may be read as the symmetric travel-time matrix obtained by dividing every distance by the common speed.

A *trip* begins at the base, visits one or more events, and returns to the base. Its elapsed duration, including travel, filming, and waiting away from the base, may not exceed the battery endurance $L > 0$. Recharge at the base is instantaneous. Trips are performed sequentially by the same drone, and waiting at the base is allowed. A trip that visits no event is called *empty*; it can always be deleted without changing the objective, so we model only *useful* trips.

At a visited event, the drone may choose any nonnegative filming duration contained in the event’s active window. The objective is to maximize the total filmed time, counting each instant of each event at most once. This last convention prevents the artificial duplication of value by assigning the same event-time interval to two trips.

3 The time-expanded graph approach

The algorithm of Caraballo et al. [1] addresses moving filming tasks. Each task is described by time-stamped waypoints (p, t) . Given a discretization step $\alpha > 0$, consecutive waypoints are subdivided so that task pieces have temporal length at most α . These time-stamped waypoints become vertices of a directed graph.

The construction has three main types of arcs:

1. *filming arcs* connect consecutive waypoints of the same task and receive a reward equal to the filming time represented by the arc;
2. *transfer arcs* connect a task waypoint to the earliest reachable waypoint of another task and receive zero filming reward; and
3. *base arcs* represent departures from, returns to, and waiting at a base station.

Every arc moves forward in time, so the graph is acyclic. A base-to-base path corresponds to a flight plan if no interval spent away from a base exceeds the battery capacity. The one-drone problem on this discretization is solved by dynamic programming over graph vertices and battery states. For n tasks, m base stations, battery capacity b , and the notation of [1], the reported running time is

$$O\left(b \left(\frac{mn}{\alpha}\right)^2 \sum_{i=1}^n |T_i|\right).$$

Thus a smaller α increases both temporal accuracy and computational cost. For multiple UAVs, the authors repeatedly solve the one-drone problem and remove the filming arcs already assigned, giving a greedy rather than globally optimal multi-drone method.

3.1 Why a polynomial algorithm can still be costly to deploy

Polynomial complexity is an asymptotic statement about a fully specified algorithm, not a measure of development effort or adaptability. In this application the dynamic program is closely matched to a particular state description. For example:

- a lower or upper bound on the number of distinct events visited requires augmenting the state or carrying additional labels;
- event-dependent rewards, mandatory coverage, setup times, or heterogeneous battery use can alter feasibility and dominance rules;
- mutual-exclusion or logical constraints couple decisions that the original recurrence treats locally;
- changing the discretization can require rebuilding a large portion of the graph; and
- each extension requires a new correctness argument and careful implementation of state transitions, pruning, and path reconstruction.

None of these points diminishes the value of the graph algorithm. Rather, they motivate a complementary approach. When the number of events and the number of relevant time pieces are moderate, an exact MILP can be a better engineering choice: new linear requirements usually become new variables or constraints, mature solvers provide bounds and optimality gaps, and the formulation supplies an independent benchmark for tailored heuristics.

4 Structural bounds and event splitting

Two quantities are not known a priori: how many base-to-base trips the drone will use and how many times it can meaningfully re-enter an event. We now derive finite conservative bounds for both.

4.1 A bound on the number of useful trips

Define

$$d_0 := \min_{i \in \mathcal{K}} D_{0i}.$$

We assume $d_0 > 0$; an event located at the base can be handled separately without a flight.

Proposition 4.1 (Minimum duration of a useful trip). *Every useful base-to-base trip has elapsed duration at least $2d_0$.*

Proof. Consider a useful trip and let i be any event visited by it. Before filming at i , the drone travels from the base to i , possibly through other events; after its last visit to i , it returns to the base, again possibly through other events. By the triangle inequality, the travel time of the prefix from 0 to i is at least D_{0i} , and the travel time of the suffix from i to 0 is at least $D_{i0} = D_{0i}$. Filming and waiting times are nonnegative. Hence the total duration is at least $2D_{0i} \geq 2d_0$. $\square$

Corollary 4.2 (Safe trip bound). *The number R^* of useful trips in any schedule contained in $[0, T]$ satisfies*

$$R^* \leq R_{\max} := \left\lfloor \frac{T}{2d_0} \right\rfloor.$$

Proof. Trips of a single drone do not overlap. By [theorem 4.1](#), their combined duration is at least $2R^*d_0$, while it cannot exceed T . Thus $2R^*d_0 \leq T$ and the result follows because R^* is integral. $\square$

This is deliberately a generous bound: service and waiting make most useful trips longer than $2d_0$. It is nevertheless safe and removes the need to guess the number of trips. For a particular instance more careful analysis of the events can yield better bounds, for instance if d_0 is given by a node k , whose active filming window is at the beginning, then the minimum trip from time t_k^* might have a bigger value. Though, in general we cannot get better bounds than this, since in some extremal cases this bound on trips is actually reached.

4.2 Minimum return excursions

For an event i , define the length of the shortest nontrivial excursion that leaves i , visits some other modeled location, and returns to i :

$$\delta_i := 2 \min_{j \in (\{0\} \cup \mathcal{K}) \setminus \{i\}} D_{ij}.$$

Including the base is essential: a drone may leave i , replace its battery, and later return.

Proposition 4.3 (Minimum duration of a genuine re-entry). *Suppose the drone leaves event i , visits at least one different modeled location, and later returns to i . The time between departure from and return to i is at least δ_i .*

Proof. Let $j \neq i$ be any other location visited during the excursion. Split the excursion at a visit to j . By the triangle inequality, the portion from i to j takes at least D_{ij} and the remaining portion from j back to i takes at least $D_{ji} = D_{ij}$. Therefore the excursion lasts at least $2D_{ij} \geq \delta_i$. $\square$

Corollary 4.4 (One genuine visit per short slice). *Let $J \subseteq I_i$ be an interval with $|J| < \delta_i$. A feasible schedule cannot contain two filming periods at event i inside J separated by a genuine excursion to another modeled location.*

Proof. Such two periods would require the drone to depart from and return to i within an elapsed time strictly smaller than δ_i , contradicting [theorem 4.3](#). $\square$

The result gives a physical reason for slicing a long event window into pieces of length at most δ_i : within the interior of one such piece, multiple separated visits cannot be useful. Equality cases can be removed by using a slightly shorter slice, or treated by a consistent half-open interval convention. Importantly, [theorem 4.3](#) is exact, while the choice of a regular partition is a modeling device. A fixed partition restricts the times at which the model distinguishes separate visits; using a finer partition improves temporal fidelity at the price of a larger MILP.

4.3 Split nodes

Choose a slice length $h_i \in (0, \delta_i]$ and let

$$N_i = \left\lceil \frac{t_i^* - t_i}{h_i} \right\rceil.$$

For $\ell = 1, \dots, N_i$, create a subnode $w = (i, \ell)$ with window

$$[a_w, b_w] = [t_i + (\ell - 1)h_i, \min\{t_i + \ell h_i, t_i^*\}]$$

and cap $\bar{f}_w = b_w - a_w$. The node set is

$$\mathcal{N} = \{0\} \cup \{(i, \ell) : i \in \mathcal{K}, \ell = 1, \dots, N_i\}.$$

For a subnode $w = (i, \ell)$, write $\pi(w) = i$ for its physical location and set $\tau_{uv} = D_{\pi(u), \pi(v)}$ when $u, v \neq 0$, with the natural definitions for the base.

The directed arc set $\mathcal{A}$ contains candidate transitions that can be feasible. We retain base-to-subnode and subnode-to-base arcs when the battery permits them. Between different events, (u, v) is retained only if

$$a_u + \tau_{uv} \leq b_v \quad \text{and} \quad D_{0, \pi(u)} + \tau_{uv} + D_{\pi(v), 0} \leq L,$$

with stronger tests available when service caps are included. Between subnodes of the same event we retain only forward arcs. For consecutive pieces these arcs have travel time zero and represent continuous filming or waiting on site.

5 MILP formulation

Let $\mathcal{R} = \{1, \dots, R\}$ with $R = R_{\max}$ or any smaller valid operational bound. For a node u , let $\Gamma^+(u) = \{v : (u, v) \in \mathcal{A}\}$ and $\Gamma^-(u) = \{v : (v, u) \in \mathcal{A}\}$.

5.1 Variables

For each trip $r \in \mathcal{R}$ we introduce:

$x_{uv}^r \in \{0, 1\}$	$(u, v) \in \mathcal{A},$	arc usage,
$y_w^r \in \{0, 1\}$	$w \in \mathcal{N} \setminus \{0\},$	subnode visit,
$s_u^r \geq 0$	$u \in \mathcal{N},$	service start (with $s_0^r = \alpha_r$),
$f_w^r \geq 0$	$w \in \mathcal{N} \setminus \{0\},$	filming duration,
$\alpha_r, \beta_r \geq 0,$		departure and return times,
$q_r \in \{0, 1\},$		trip activation.

We define $f_0^r = 0$. The objective is

$$\max \sum_{r \in \mathcal{R}} \sum_{w \in \mathcal{N} \setminus \{0\}} f_w^r. \quad (1)$$

5.2 Routing and activation

Each active trip leaves and returns to the base once:

$$\sum_{v \in \Gamma^+(0)} x_{0v}^r = q_r, \quad r \in \mathcal{R}, \quad (2)$$

$$\sum_{u \in \Gamma^-(0)} x_{u0}^r = q_r, \quad r \in \mathcal{R}. \quad (3)$$

At every subnode, incoming and outgoing flow equal the visit indicator:

$$\sum_{u \in \Gamma^-(w)} x_{uw}^r = y_w^r, \quad r \in \mathcal{R}, \quad w \neq 0, \quad (4)$$

$$\sum_{v \in \Gamma^+(w)} x_{wv}^r = y_w^r, \quad r \in \mathcal{R}, \quad w \neq 0, \quad (5)$$

$$y_w^r \leq q_r, \quad r \in \mathcal{R}, \quad w \neq 0. \quad (6)$$

5.3 Windows, service, and temporal precedence

For every $r \in \mathcal{R}$ and $w \neq 0$,

$$a_w y_w^r \leq s_w^r \leq b_w y_w^r, \quad (7)$$

$$s_w^r + f_w^r \leq b_w + b_w(1 - y_w^r), \quad (8)$$

$$0 \leq f_w^r \leq \bar{f}_w y_w^r. \quad (9)$$

The extra deactivation term in (8) makes the intended logic explicit; with (7) and $f_w^r = 0$ when unvisited, it may be omitted.

For each $(u, v) \in \mathcal{A}$ with $v \neq 0$,

$$s_v^r \geq s_u^r + f_u^r + \tau_{uv} - M_{uv}(1 - x_{uv}^r), \quad r \in \mathcal{R}. \quad (10)$$

Valid arc-specific constants are preferable to one global constant. With $\underline{s}_0 = 0$, $\bar{s}_0 = T$, $\bar{f}_0 = 0$, and $\underline{s}_w = a_w$, $\bar{s}_w = b_w$ for subnodes, one safe choice is

$$M_{uv} = \max\{0, \bar{s}_u + \bar{f}_u + \tau_{uv} - \underline{s}_v\}. \quad (11)$$

Whenever an arc can be ruled out by windows or battery before optimization, deleting it is stronger than assigning it a large M_{uv} .

5.4 Trip timing, battery, and sequencing

For every $r \in \mathcal{R}$,

$$\alpha_r = s_0^r, \tag{12}$$

$$0 \leq \alpha_r \leq Tq_r, \tag{13}$$

$$\beta_r \geq s_u^r + f_u^r + \tau_{u0} - M_{u0}(1 - x_{u0}^r), \quad u \in \Gamma^-(0), \tag{14}$$

$$0 \leq \beta_r \leq Tq_r, \tag{15}$$

$$\beta_r - \alpha_r \leq Lq_r. \tag{16}$$

Since inactive trips have $\alpha_r = \beta_r = 0$, the stronger right-hand side Lq_r can be used in place of a deactivated big- M battery constraint.

Trips are ordered and packed at the beginning of the index set:

$$\alpha_{r+1} \geq \beta_r - T(1 - q_{r+1}), \quad r = 1, \dots, R-1, \tag{17}$$

$$q_{r+1} \leq q_r, \quad r = 1, \dots, R-1. \tag{18}$$

Constraint (18) is a symmetry breaker as well as a convenient convention.

5.5 Preventing duplicate value

Each slice may be assigned to at most one trip:

$$\sum_{r \in \mathcal{R}} y_w^r \leq 1, \quad w \in \mathcal{N} \setminus \{0\}. \tag{19}$$

This is exact for the split-node interpretation in which one subnode represents one connected service period. If the application should allow several trips to film disjoint parts of a relatively coarse slice, the correct remedy is to refine that slice, not to drop (19) and risk double counting.

6 Validity of the formulation

Flow balance alone can admit disconnected directed cycles. The time variables eliminate all cycles with positive accumulated duration.

Proposition 6.1 (Temporal constraints eliminate positive-time subtours). *Fix a trip r . Suppose every directed cycle in the retained graph contains an arc or a visited node with strictly positive travel or service time. Then (10) forbids any used directed cycle not containing the base.*

Proof. Assume that a used cycle $C = (u_1, u_2, \dots, u_m, u_1)$ exists. Since $x_{u_j u_{j+1}}^r = 1$, summing (10) around the cycle gives

$$\sum_{j=1}^m s_{u_{j+1}}^r \geq \sum_{j=1}^m s_{u_j}^r \sum_{j=1}^m (f_{u_j}^r + \tau_{u_j u_{j+1}}).$$

The two sums of start times are identical, so

$$0 \geq \sum_{j=1}^m (f_{u_j}^r + \tau_{u_j u_{j+1}}),$$

contradicting nonnegativity and the assumed strict positivity on the cycle. $\square$

The only zero-travel transitions in the standard construction are those between subnodes representing the same physical event. These arcs are retained only in the forward direction of the event partition. Consequently, they cannot form a directed cycle. Assuming distinct physical locations have strictly positive pairwise travel times, every directed cycle in A therefore contains a positive-travel arc and is eliminated by Proposition 6.1. Hence no additional subtour-elimination constraints or cycle-removal procedure is required.

Theorem 6.2 (Exactness of the MILP formulation). *Assume that D is a metric, that $h_i \leq \delta_i$ for every $i \in K$, and that R is a valid upper bound on the number of useful trips. Suppose that A contains all feasible transitions required by the split-node construction, with arcs between subnodes of the same event directed forward, and that the constants in (10)–(14) are valid. Then:*

- (a) *every feasible integer solution of (2)–(19) induces a feasible continuous-time schedule with at most R useful trips and the same total filming time;*
- (b) *every feasible continuous-time schedule can be transformed, without changing its total filming time, into a schedule represented by a feasible integer solution.*

Consequently, (1) has the same optimal value as the original continuous-time problem.

Proof. For (a), fix an active trip r . Equations (2) and (3) give one departure and one return arc, while (4)–(5) give each visited subnode exactly one predecessor and one successor. Hence the used arcs consist of a base-to-base path plus, potentially, disjoint directed cycles. A cycle involving different physical locations has positive travel time and is excluded by theorem 6.1. A cycle involving only subnodes of one event is impossible because their indices strictly increase along every retained arc.

For each visited subnode w , let

$$H_w^r = [s_w^r, s_w^r + f_w^r].$$

Constraints (7)–(9) and (10) imply

$$H_w^r \subseteq [a_w, b_w], \quad s_v^r \geq s_u^r + f_u^r + \tau_{uv} \quad \text{if } x_{uv}^r = 1.$$

These filming intervals can therefore be realized in path order. If u is the last subnode, then (14) and (16) give

$$\widehat{\beta}_r := s_u^r + f_u^r + \tau_{u0} \leq \beta_r, \quad \widehat{\beta}_r - \alpha_r \leq \beta_r - \alpha_r \leq L.$$

Thus the physical return occurs within the battery limit. Constraints (17)–(18) order the trips chronologically, and (19) assigns each slice to at most one trip. Since slices of the same event have disjoint interiors, no filming duration is counted twice. Discarding trips with zero filming time yields the required schedule, with total filming time

$$\sum_{r \in \mathcal{R}} \sum_{w \in N \setminus \{0\}} f_w^r.$$

For (b), first normalize a feasible continuous-time schedule. Discard trips with zero filming time and shortcut nonfilming visits, retaining necessary returns to the base. The triangle inequality ensures that travel time does not increase; saved time may be replaced by waiting. During each uninterrupted stay at an event, consolidate its filming periods into one interval of the same total duration within the stay and the event window. Excursions visiting no other modeled location may likewise be replaced by waiting at the event. These operations preserve feasibility and total filming time.

Fix a slice $J = [a_w, b_w]$ of event i . Suppose that two distinct visits contribute positive filming time to J . Let d be the departure after the first visit and e the return before the second. The minimum-excursion bound gives

$$e - d \geq \delta_i,$$

whereas positive filming within J before departure and after return implies

$$a_w < d \leq e < b_w.$$

Hence

$$\delta_i \leq e - d < b_w - a_w \leq h_i \leq \delta_i,$$

a contradiction. Thus each slice receives positive filming time from at most one visit, and hence from at most one trip.

Intersect each filming interval with the slices of its event, discarding intersections of zero duration. Each remaining portion is assigned to its corresponding subnode. Portions crossing a breakpoint are connected by forward zero-travel arcs; all other required transitions belong to $\mathcal{A}$ by assumption.

Order the trips chronologically and assign them indices $1, \dots, R'$, with $R' \leq R$. Set $q_r = 1$ for $r \leq R'$ and zero otherwise. For each active trip, set x^r to its path arcs, y^r to its used subnodes, s^r, f^r to the starts and durations of its filming portions, and α_r, β_r to its departure and return times. Set variables of unvisited subnodes and inactive trips to zero.

The routing, timing, battery, and sequencing constraints follow from schedule feasibility and validity of the deactivation constants. The preceding argument gives

$$\sum_{r \in \mathcal{R}} y_w^r \leq 1 \quad \text{for every } w \neq 0,$$

which is (19). Finally, for every filming interval H at event i ,

$$|H| = \sum_{\ell=1}^{N_i} |H \cap [a_{(i,\ell)}, b_{(i,\ell)}]|,$$

so the objective is unchanged. Together, (a) and (b) establish equality of the optimal values. $\square$

Remark 6.3 (Role of the event partition). Theorem 6.2 establishes exactness for the original continuous-time problem. The partition does not discretize filming start times or durations, which remain continuous variables. Rather, $h_i \leq \delta_i$ ensures that one subnode per slice suffices after normalization.

The formulation need not reproduce every feasible schedule literally, but represents a schedule of the same value as every feasible continuous-time schedule. In particular, an optimal schedule is retained whenever an optimum exists. If slices longer than δ_i are used, distinct visits may film disjoint portions of the same slice; constraint (19) may then exclude such schedules, and exactness is no longer guaranteed.

7 Flexibility of the MILP approach

The main practical advantage of the formulation is that many application-specific demands can be introduced without redesigning the entire solution method.

7.1 Rewarding or requiring event diversity

Let $z_i \in \{0, 1\}$ indicate whether event i is filmed. The exact logical link is

$$z_i \geq y_{(i,\ell)}^r, \quad i \in \mathcal{K}, \ell = 1, \dots, N_i, r \in \mathcal{R}, \quad (20)$$

$$z_i \leq \sum_{r \in \mathcal{R}} \sum_{\ell=1}^{N_i} y_{(i,\ell)}^r, \quad i \in \mathcal{K}. \quad (21)$$

A minimum-diversity requirement is

$$\sum_{i \in \mathcal{K}} z_i \geq G,$$

whereas a prize-collecting objective is

$$\max \left\{ \sum_{r \in \mathcal{R}} \sum_{w \neq 0} f_w^r + \lambda \sum_{i \in \mathcal{K}} z_i \right\}, \quad \lambda \geq 0.$$

The parameter λ makes the trade-off explicit: a larger value may sacrifice some filming time in exchange for covering more distinct events.

7.2 Other linear extensions

Examples include event weights c_i in the filming objective; minimum filming amounts $\sum_{r,\ell} f_{(i,\ell)}^r \geq m_i z_i$; mandatory events $z_i = 1$; a limit on the number of trips $\sum_r q_r \leq \bar{R}$; fixed recharge times $\alpha_{r+1} \geq \beta_r + c - T(1 - q_{r+1})$; trip activation costs; and precedence or mutual-exclusion rules between events. More detailed energy models can replace elapsed time by arc-specific energy consumption, provided the relevant resource accumulation is modeled linearly.

These modifications illustrate the central comparison. In a dynamic program, an extension that depends on history may require a new state coordinate, causing a multiplicative growth in the state space and a new proof of the recurrence. In a MILP, the same requirement often appears as a small block of constraints. This convenience does not make the MILP computationally free; it shifts effort from algorithm design to model size and relaxation quality.

8 Illustrative instances

We complement the formulation with a small collection of deterministic instances intended to illustrate characteristic behaviours of the model. These examples are not designed as a computational benchmark; rather, they serve as implementation checks and illustrate how the optimal schedule responds to different temporal structures and modeling requirements.

8.1 An instance attaining the trip bound

We first illustrate that the general trip bound of Corollary 4.2 can be attained.

Consider $K = 10$ events over the planning horizon

$$[0, T] = [0, 21].$$

Every event is at distance one from the base,

$$D_{0i} = 1, \quad i = 1, \dots, 10,$$

while any two distinct event locations are at distance

$$D_{ij} = 2, \quad i \neq j.$$

Therefore

$$d_0 = \min_{i \in K} D_{0i} = 1,$$

and Corollary 4.2 gives

$$R_{\max} = \left\lfloor \frac{T}{2d_0} \right\rfloor = \left\lfloor \frac{21}{2} \right\rfloor = 10.$$

Each event has a window of length 0.1, with starting times

$$1, 3.1, 5.2, \dots, 19.9.$$

The battery endurance is $L = 4$.

The windows are arranged so that each event is optimally served by a different base-to-base trip. A trip filming one complete event consists of one unit of outward travel, 0.1 units of filming, and one unit of return travel. Hence each trip occupies 2.1 units of time, and all ten trips fit exactly within the horizon:

$$10 \cdot 2.1 = 21.$$

Thus

$$R^* = 10 = R_{\max}.$$

This shows that the bound of Corollary 4.2 is sharp in general: without exploiting additional information about the temporal structure of the events, the universal bound cannot be reduced.

Figure 1 reproduces the structure of the instance. As in the presentation, the horizontal scale is normalized for readability.

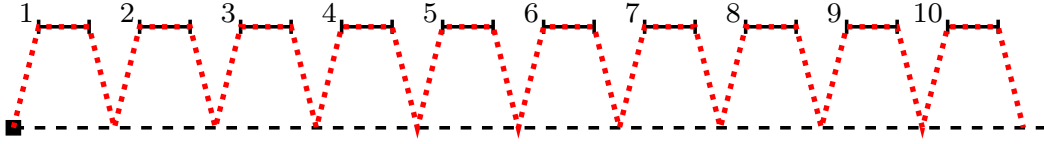


Figure 1: Instance attaining the upper bound on the number of useful trips. The optimal trajectory is shown in red.

8.2 An instance with overlapping event windows

We next consider a less structured instance containing five events with different and partially overlapping time windows. The planning horizon is $T = 90$ and the battery endurance is $L = 14$. The event windows are

$$\begin{aligned} I_1 &= [12, 60], & I_2 &= [20, 44], \\ I_3 &= [52, 68], & I_4 &= [68, 88], \\ I_5 &= [72, 80]. \end{aligned}$$

The distances from the base are

$$(D_{01}, D_{02}, D_{03}, D_{04}, D_{05}) = (4, 3, 3, 4, 4),$$

and the symmetric inter-event distance matrix is

$$D = \begin{pmatrix} 0 & 4 & 5 & 5 & 5 \\ 4 & 0 & 3 & 4 & 5 \\ 5 & 3 & 0 & 3 & 3 \\ 5 & 4 & 3 & 0 & 2 \\ 5 & 5 & 3 & 2 & 0 \end{pmatrix}.$$

For the split-node construction we use

$$h_1 = h_2 = h_3 = 6, \quad h_4 = h_5 = 4.$$

In contrast with the preceding construction, several filming opportunities now compete for overlapping portions of the planning horizon. The model must decide jointly which events to visit, for how long they should be filmed, and how the selected filming intervals should be divided among battery-feasible trips.

The resulting optimal schedule is represented in Figure 2. The graphic follows the same convention as the presentation: event windows are shown in black and the optimal drone trajectory is shown in red.

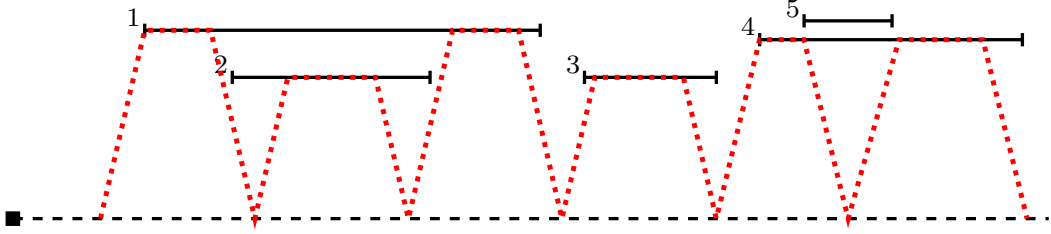


Figure 2: Optimal schedule for the five-event instance with overlapping windows and heterogeneous travel times. The optimal trajectory is shown in red.

8.3 Rewarding event diversity

The same type of instance also illustrates the flexibility of the MILP formulation. Let $z_i \in \{0, 1\}$ indicate whether event i is filmed and consider the modified objective

$$\max \left\{ \sum_{r \in \mathcal{R}} \sum_{w \in \mathcal{N} \setminus \{0\}} f_w^r + \lambda \sum_{i \in K} z_i \right\}, \quad \lambda \geq 0.$$

For $\lambda = 0$, only total filming time contributes to the objective. When $\lambda > 0$, filming an additional distinct event provides an extra reward and may therefore compensate for a moderate loss in total filming time.

Figure 3 compares the two schedules using the same colour convention as in the presentation. The original optimum with $\lambda = 0$ is shown by the red dotted trajectory, while the solution with $\lambda = 3$ is shown by the solid blue trajectory.

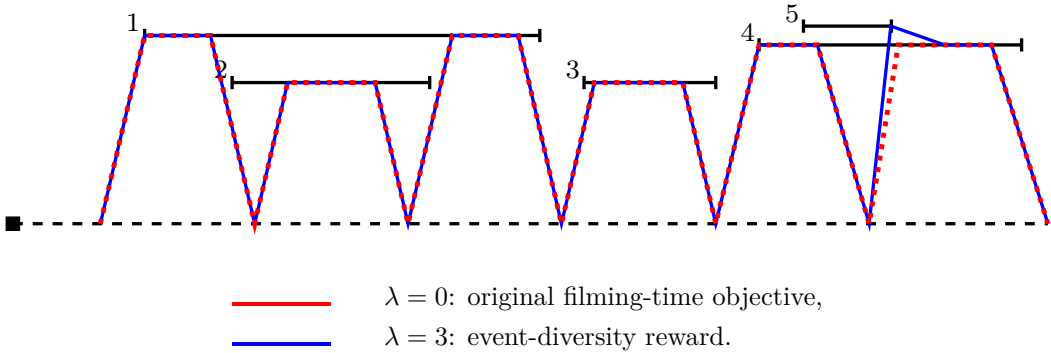


Figure 3: Effect of the prize-rewarding extension. The red dotted route corresponds to $\lambda = 0$, while the solid blue route corresponds to $\lambda = 3$.

The comparison illustrates the qualitative effect of the extension. With $\lambda = 0$, the model optimizes filming time alone. Once event diversity is rewarded, the optimizer may reallocate part of the available filming time in order to include an event that was previously omitted. Importantly, the routing, timing, battery, and split-node constraints are unchanged; only the additional visit variables, their linking constraints, and the modified objective are required.

These examples highlight two complementary aspects of the formulation. The extremal instance shows that the general trip bound can be attained, whereas the overlapping-window and prize-rewarding examples illustrate how the same MILP accommodates more realistic temporal interactions and additional coverage criteria without redesigning the underlying routing model.

9 Computational considerations

Let $N = |\mathcal{N}| - 1 = \sum_i N_i$ and $A = |\mathcal{A}|$. The formulation contains $O(RA)$ binary arc variables and $O(RN)$ visit binaries and continuous timing variables. Consequently, the number of feasible arcs is often a better predictor of difficulty than the number of physical events alone. Long windows create many subnodes; temporal overlap creates many reachable transitions; and the conservative trip bound replicates these variables for many trip indices.

The following practices are therefore important:

1. remove arcs that violate time reachability, return-to-base feasibility, or the battery limit;
2. use arc-specific M_{uv} values from (11);
3. keep only forward, preferably consecutive, same-event arcs;
4. use the smallest operationally valid trip bound, while retaining [theorem 4.2](#) as a safe fallback;
5. choose event partitions adaptively: finer near relevant window boundaries and coarser where no competing event is reachable; and
6. warm-start the MILP with a schedule produced by a greedy or dynamic-programming heuristic when repeated solution is required.

Our exploratory R-Gurobi implementation exhibits the expected pattern in randomly generated small instances: solve time grows moderately for many sparse instances but can rise sharply when the reachable-arc graph becomes dense. [Figure 4](#) is therefore best interpreted as an implementation check and a motivation for pruning, not as a solver-independent complexity law. Reproducible computational claims would require the instance generator, parameter ranges, hardware, solver version, tolerances, seeds, and raw timing data to accompany the experiment.

10 When should each approach be used?

The graph algorithm and the MILP are complementary rather than competing in every regime. [Table 1](#) summarizes the practical distinction.

For small instances, the MILP can thus be preferable even though a polynomial specialized algorithm exists: the relevant cost is the combined cost of modeling, implementation, validation, maintenance, and computation. For large-scale deployment, the natural hybrid is to retain the specialized method or a heuristic as the operational solver and use the MILP on reduced instances to validate solution quality, tune pruning rules, and test new requirements before incorporating them into bespoke code.

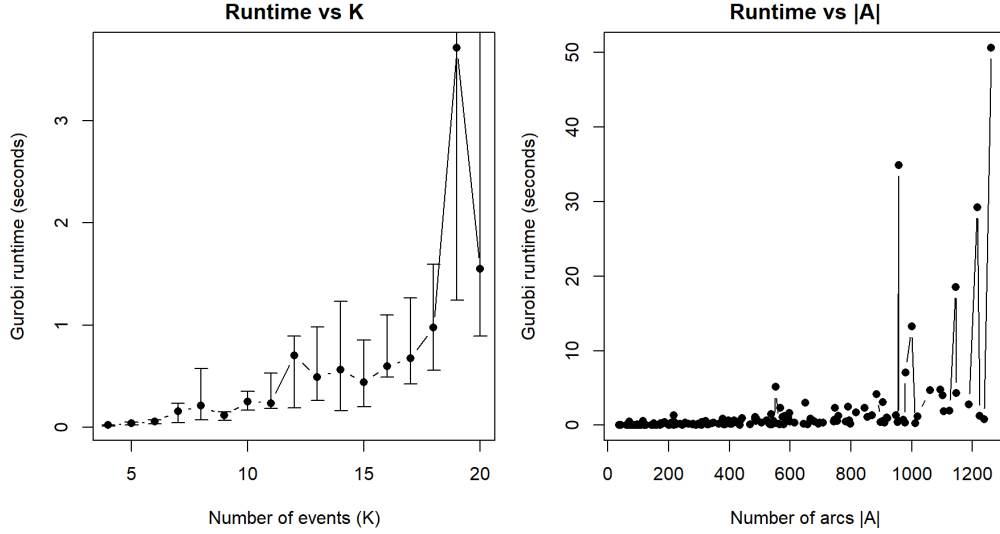


Figure 4: Exploratory Gurobi solve times for randomly generated instances, displayed against the number of events K and the number of retained arcs $|\mathcal{A}|$. The plot is retained from the preliminary implementation and is descriptive rather than a formal benchmark.

Table 1: Graph dynamic programming versus the proposed MILP.

Criterion	Time-expanded graph / DP	MILP
Core guarantee	Polynomial solution of the stated discretized one-drone problem	Exact solution and optimality gap for the chosen split-node model
Worst-case scaling	Polynomial in the expanded graph and battery representation	Exponential in general because of integrality
Implementation	Bespoke graph, state, recurrence, dominance, and reconstruction	Standard variables and constraints handled by a general-purpose solver
New requirements	May require new state variables and correctness proofs	Often added as linear variables or constraints
Best regime	Large or repeatedly solved, stable problem family	Small/medium, changing, constraint-rich instances; exact benchmarking
Main risk	State-space and discretization growth	Branch-and-bound growth and weak big- M relaxations

11 Conclusion

We presented a rigorous finite MILP model for maximizing the filming time of a battery-limited drone over events with time windows. The formulation handles an unknown number of trips through a proved trip bound, represents meaningful re-entry through physically justified event splitting, and uses temporal variables both for scheduling and subtour elimination. The main correction to an informal treatment is that the trip bound uses a floor and the round-trip distance $2D_{0i}$, while the splitting threshold is twice the distance to the nearest different modeled location and should include the base whenever recharge-and-return is possible.

The model is not proposed as a universal replacement for graph-based dynamic programming. Its value lies in a different regime: small and medium instances for which requirements are still evolving, exact certificates are useful, and implementation flexibility matters. A natural next step is a reproducible computational study comparing the MILP, the time-expanded dynamic program, and greedy heuristics on a common family of instances, with controlled variation of temporal overlap, partition granularity, arc density, and battery endurance.

References

- [1] L.-E. Caraballo, À. Montes-Romero, J.-M. Díaz-Báñez, J. Capitán, A. Torres-González, and A. Ollero. Autonomous planning for multiple aerial cinematographers. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1509–1515, 2020. doi: 10.1109/IROS45743.2020.9341622.
- [2] O. Aichholzer, L.-E. Caraballo, J.-M. Díaz-Báñez, R. Fabila-Monroy, I. Ventura, and B. Vogtenhuber. Scheduling aerial robots to cover outdoor events. In *Proceedings of the 36th European Workshop on Computational Geometry (EuroCG)*, 2020.
- [3] J.-M. Díaz-Báñez and R. Fabila-Monroy. Filming runners with drones is hard. *Theoretical Computer Science*, 1082:116129, 2026. doi: 10.1016/j.tcs.2026.116129.
- [4] P. Toth and D. Vigo, editors. *Vehicle Routing: Problems, Methods, and Applications*. SIAM, second edition, 2014.
- [5] L. A. Wolsey. *Integer Programming*. Wiley, 1998.